

Introdução à Computação

bubble sort

Alexandre Rademaker

Ordenação (sort) I

Vamos considerar dois algoritmos para esta tarefa:

- ▶ selection sort
- ▶ bubble sort
- ▶ merge sort

Bubble Sort I

```
1 Repeat n - 1 times
2   For i from 0 to n - 2
3     If numbers[i] and numbers[i + 1] out of order
4       Swap them
```

No **bubble sort** vamos realizar trocas sucessivas de elementos adjacentes até que nenhuma troca seja possível.

Por que o loop mais externo é de $n - 1$ passos?

Bubble Sort II

1	5	2	7	4	1	6	3	0
2	^	^						
3	2	5	7	4	1	6	3	0
4		^	^					
5	2	5	7	4	1	6	3	0
6			^	^				
7	2	5	4	7	1	6	3	0
8				^	^			
9	2	5	4	1	7	6	3	0
10					^	^		
11	2	5	4	1	6	7	3	0
12						^	^	
13	2	5	4	1	6	3	7	0
14							^	^
15	2	5	4	1	6	3	0	7

Bubble Sort III

Agora o maior número está no final da lista, e podemos continuar

1	2	5	4	1	6	3	0		7
2	^	^							
3	2	5	4	1	6	3	0		7
4		^	^						
5	2	4	5	1	6	3	0		7
6			^	^					
7	2	4	1	5	6	3	0		7
8				^	^				
9	2	4	1	5	6	3	0		7
10					^	^			
11	2	4	1	5	3	6	0		7
12						^	^		
13	2	4	1	5	3	0		6	7

A cada passada, a lista de números que precisamos examinar é menor.

Bubble Sort IV

bubble.c

```
1 void bubble_sort (int a[], int n) {
2     int i, t, j = n;
3     bool s = true;
4
5     while (s) {
6         s = false;
7         for (i = 1; i < j; i++) {
8             if (a[i] < a[i - 1]) {
9                 t = a[i];
10                a[i] = a[i - 1];
11                a[i - 1] = t;
12                s = true;
13            }
14        }
15        j--;
16    }
17 }
```

Bubble Sort V

Epa! Tem alguma coisa diferente em relação ao pseudo-código! O que?

Ao invés de passar $n - 1$ vezes na lista, passamos enquanto trocas forem feitas!

A troca de valores entre duas posições de um array está ficando repetitiva, não?! Vamos falar disso depois...

Bubble Sort VI

Sobre complexidade? Na versão original

$$\begin{aligned}(n-1) \times (n-1) &= \\ n^2 - 2n + 1 &= O(n^2)\end{aligned}$$

e também $\Omega(n^2)$.

Mas com nossa otimização, reduzimos o melhor caso (lista já ordenada) para $\Omega(n)$.